

```

1  import os
2  import sys
3
4  sys.path.append(os.path.join(os.path.dirname(__file__), ".."))
5
6  import matplotlib.pyplot as plt
7  import numpy as np
8  import pandas as pd
9  import shap
10 import streamlit as st
11 from configs.config import NDF4W_1P5C_OSS_PATH, NDF4W_1P5C_LOCAL_PATH
12 from configs.logging import logger
13 from connection.connection import DBConnector
14 from mappings.mappings import mappings
15 from utils.util import load_data
16 from io import BytesIO
17
18 prefix_table = "ndf4w_1p5c_"
19
20 feature_group_map_cols = mappings["feature_group_map_cols"]
21 rename_feature = mappings["rename_feature"]
22 ids_col = mappings["columns"]["idsinfos"]
23 overlays_col = mappings["columns"]["overlayinfos"]
24 feature_npl = mappings["columns"]["feature_npl"]
25 feature_non_npl = mappings["columns"]["feature_non_npl"]
26
27
28 def clean_feature_name(feature):
29     replacements = {
30         '_v1': '',
31         'bpkb_owner_status_': 'bpkb_owner_status',
32         'tenor_': 'tenor',
33     }
34
35
36     for old, new in replacements.items():
37         feature = feature.replace(old, new)
38
39     if feature.startswith('pef_'):
40         feature = feature.replace('pef_', '', 1)
41
42     return feature
43
44
45 def reshape_data(data: pd.DataFrame, cols: list, var_name: str, value_name: str) -> pd.
46 DataFrame:
47     """Transform a DataFrame into long format."""
48
49     return (
50         data
51         .filter(cols)
52         .T
53         .reset_index()
54         .rename(columns={'index': var_name, 0: value_name})
55     )
56
57 def get_id_data(data: pd.DataFrame) -> pd.DataFrame:
58     """Extract ID-related information."""
59
60     id_col = [
61         'lead_id_feat', 'application_id_res', 'created_at_logfeat',
62         'created_at_logres', 'created_at_ovly_p5c', 'model_category_pd',
63         'branch', 'full_name'
64     ]
65
66     return reshape_data(data, id_col, 'variable', 'value')

```

```

67
68
69 def get_feature_data(data: pd.DataFrame) -> pd.DataFrame:
70     """Extract and merge feature-related data dynamically."""
71
72     feature_col = ['dsr', 'bpkb_owner_status', 'unit_type_area',
73                   'asset_age_unit_type', 'asset_age_asset_usage']
74
75     score_col = [f"{feature}_score" for feature in feature_col]
76     max_score_col = [f"{feature}_max_score" for feature in feature_col]
77     min_score_col = [f"{feature}_min_score" for feature in feature_col]
78     percent_col = [f"{feature}_percent" for feature in feature_col]
79
80     data_feature = reshape_data(data, feature_col, 'variable', 'value')
81     data_score = reshape_data(data, score_col, 'variable_2', 'score')
82     data_max_score = reshape_data(data, max_score_col, 'variable_3', 'max_score')
83     data_min_score = reshape_data(data, min_score_col, 'variable_4', 'min_score')
84     data_percent = 100*(reshape_data(data, percent_col, 'variable_5', 'percent')).round(2)
85
86     return (
87         pd.concat([data_feature, data_score, data_max_score, data_min_score, data_percent
88                   ], axis=1)
89         .drop(columns=['variable_2', 'variable_3', 'variable_4', 'variable_5'])
90         .assign(
91             feature_category=lambda x: np.where(x['variable'].isin(['dsr']), 'capacity',
92                                                'collateral')
93         )
94         .sort_values('percent', ascending=False)
95     )
96
97 def get_summary_data(data: pd.DataFrame) -> pd.DataFrame:
98     """Summarize feature category scores."""
99
100     return (
101         data
102         .groupby(['feature_category'])
103         .agg(
104             score_total=('score', 'sum'),
105             max_score_total=('max_score', 'sum'),
106             min_score_total=('min_score', 'sum')
107         )
108         .reset_index()
109         .assign(
110             percent_score=lambda x: (100 * ((x['score_total'] - x['min_score_total']) /
111                                             (x['max_score_total'] - x['min_score_total'])))
112         )
113         .sort_values('percent_score', ascending=False)
114     )
115
116
117 def get_overlay_data(data: pd.DataFrame) -> pd.DataFrame:
118     """Extract overlay-related data."""
119
120
121     overlay_col = [
122         'p5c_score', 'ltv', 'nstdetectresult', 'takeover_status',
123         'assetvalidation', 'decision_pd_p5c', 'decision_ovly_p5c',
124         'ltv_final', 'alternativetype', 'marketingid'
125     ]
126
127     return reshape_data(data, overlay_col, 'variable', 'value')
128
129
130 def process_data(data: pd.DataFrame, application_id: str) -> dict:

```

```

131     """Main function to process all data components separately."""
132
133     data_detail = data.query("application_id_res == @application_id").reset_index(drop=
True)
134
135     return {
136         "data_id": get_id_data(data_detail),
137         "data_feature": get_feature_data(data_detail),
138         "data_summary": get_summary_data(get_feature_data(data_detail)),
139         "data_overlay": get_overlay_data(data_detail)
140     }
141
142     @st.cache_data(ttl=1800)
143     def load_data_by_id(application_id):
144         pg_monitoring_connector = None
145         try:
146             pg_monitoring_connector = DBConnector("postgres_monitoring")
147         except Exception as e:
148             st.warning("⚠ Unable to connect to the monitoring database. Try loading data
from OSS instead.")
149             logger.exception(f"✗ Failed to connect to database: {e}")
150
151         str_where = None
152         if application_id:
153             str_where = f"application_id_res = '{application_id}'"
154         df = load_data(db_type="postgres_monitoring", source="raw_data_monitoring", where=
str_where, prefix_table=prefix_table, oss_path=NDF4W_1P5C_OSS_PATH, local_path=
NDF4W_1P5C_LOCAL_PATH)
155         df = df.rename(columns=lambda col: clean_feature_name(col))
156         # df = df.rename(columns=mappings["rename_general"])
157         df_shap_npl = load_data(db_type="postgres_monitoring", source=
"shap_values_ntb_npl_monitoring", where=str_where, prefix_table=prefix_table,
oss_path=NDF4W_1P5C_OSS_PATH, local_path=NDF4W_1P5C_LOCAL_PATH)
158         df_shap_non_npl = load_data(db_type="postgres_monitoring", source=
"shap_values_ntb_non_npl_monitoring", where=str_where, prefix_table=prefix_table,
oss_path=NDF4W_1P5C_OSS_PATH, local_path=NDF4W_1P5C_LOCAL_PATH)
159         feature_info_npl = load_data(db_type="postgres_monitoring", source=
"transform_data_ntb_npl_monitoring", where=str_where, prefix_table=prefix_table,
oss_path=NDF4W_1P5C_OSS_PATH, local_path=NDF4W_1P5C_LOCAL_PATH)
160         feature_info_npl = feature_info_npl.drop(columns=[col for col in feature_info_npl.
columns if 'woe' in col
161                                                         ]+['bpbk_owner_status', 'tenor',
'asset_age'])
162         feature_info_npl = feature_info_npl.rename(columns=lambda col: clean_feature_name(col
))
163         feature_info_npl = feature_info_npl.rename(columns={'asset_age': 'asset_age'})
164         feature_info_non_npl = load_data(db_type="postgres_monitoring", source=
"transform_data_ntb_non_npl_monitoring", where=str_where, prefix_table=prefix_table,
oss_path=NDF4W_1P5C_OSS_PATH, local_path=NDF4W_1P5C_LOCAL_PATH)
165         feature_info_non_npl = feature_info_non_npl.drop(columns=[col for col in
feature_info_non_npl.columns if 'woe' in col
166                                                         ]+['bpbk_owner_status',
'tenor', 'asset_age'])
167         feature_info_non_npl = feature_info_non_npl.rename(columns={'asset_age': 'asset_age'
})
168         feature_info_non_npl = feature_info_non_npl.rename(columns=lambda col:
clean_feature_name(col))
169         df_ntc = load_data(db_type="postgres_monitoring", source=
"transform_data_ntc_monitoring", where=str_where, prefix_table=prefix_table, oss_path=
NDF4W_1P5C_OSS_PATH, local_path=NDF4W_1P5C_LOCAL_PATH)
170
171         contractlist = load_data(db_type="postgres_monitoring", source=
"contractlist_exploded", where=str_where, prefix_table=prefix_table, oss_path=
NDF4W_1P5C_OSS_PATH, local_path=NDF4W_1P5C_LOCAL_PATH)
172         paymentlist = load_data(db_type="postgres_monitoring", source="paymentlist_exploded",
where=str_where, prefix_table=prefix_table, oss_path=NDF4W_1P5C_OSS_PATH, local_path=
NDF4W_1P5C_LOCAL_PATH)

```

```

173
174     return df, df_shap_npl, df_shap_non_npl, feature_info_npl, feature_info_non_npl,
df_ntc, contractlist, paymentlist
175
176
177 def agg_shap_summary(shap_info_df):
178
179     if shap_info_df.empty:
180         return None
181
182     try:
183
184         shap_grouped = shap_info_df.copy()
185
186         grouped_explanation = shap.Explanation(
187             values=shap_grouped['shap_value'].values,
188             base_values=shap_grouped['base_values'].values[0],
189             feature_names=shap_grouped['group'].values
190         )
191
192         fig, ax = plt.subplots(figsize=(25, 15), dpi=192)
193
194         shap.plots.waterfall(grouped_explanation, max_display=shap_grouped.shape[0], show
=False)
195
196         return fig
197
198     except Exception as e:
199         st.error(f"Error processing SHAP summary: {str(e)}")
200         return None
201
202 # --- Helper: Convert DataFrame to Excel in memory ---
203 def to_excel_bytes(dataframe):
204     output = BytesIO()
205     with pd.ExcelWriter(output, engine='xlsxwriter') as writer:
206         dataframe.to_excel(writer, index=False, sheet_name='Sheet1')
207     return output.getvalue()
208
209 def main(application_id):
210     if not application_id:
211         st.warning("⚠ Please enter an Application ID.")
212         return
213
214     df, df_shap_npl, df_shap_non_npl, feature_info_npl, feature_info_non_npl, df_ntc,
contractlist, paymentlist = load_data_by_id(application_id=application_id)
215
216     application_id = str(application_id)
217
218     ntb_ids = set(map(str, df[df['model_category_pd'].isin(['NTB_NPL', 'NTB_NON_NPL'])][
'application_id_res'].values))
219     ntc_ids = set(map(str, df[df['model_category_pd'].isin(['NTC'])]['application_id_res'
].values))
220
221     contractlist_bytes = to_excel_bytes(contractlist)
222     paymentlist_bytes = to_excel_bytes(paymentlist)
223
224     if application_id in ntb_ids:
225         app_info = df[df['application_id_res'].astype(str) == application_id]
226         shap_info_npl = df_shap_npl[df_shap_npl['application_id_res'].astype(str) ==
application_id]
227         shap_info_non_npl = df_shap_non_npl[df_shap_non_npl['application_id_res'].astype(
str) == application_id]
228
229         fig_npl = agg_shap_summary(shap_info_npl)
230         fig_non_npl = agg_shap_summary(shap_info_non_npl)
231
232         if fig_npl:

```

```

233         st.pyplot(fig_npl, use_container_width=True)
234     elif fig_non_npl:
235         st.pyplot(fig_non_npl, use_container_width=True)
236
237     st.subheader("Application Information")
238
239     st.subheader("📄 ID")
240
241     df = app_info.rename(columns=mappings["rename_general"])[ids_col].T.reset_index()
242     df = df.rename(columns={'index': "Detail", df.columns[1]: "Values"})
243     df["Values"] = df["Values"].astype(str)
244
245     st.dataframe(df, use_container_width=True, hide_index=True)
246
247     st.subheader("📄 Feature Information")
248     if application_id in feature_info_npl['application_id_res'].astype(str).values:
249         feature_data = feature_info_npl[feature_npl].T.reset_index()
250     else:
251         feature_data = feature_info_non_npl[feature_non_npl].T.reset_index()
252
253     feature_data = feature_data.rename(columns={'index': "feature_initial",
254         feature_data.columns[-1]: "Values"})
255     feature_data["Values"] = feature_data["Values"].astype(str)
256
257     feature_data['Feature Category'] = feature_data['feature_initial'].map(
258         feature_group_map_cols)
259     feature_data['Feature'] = feature_data['feature_initial'].map(rename_feature)
260
261     st.dataframe(
262         feature_data[['Feature', 'Feature Category', 'Values']],
263         use_container_width=True,
264         hide_index=True
265     )
266
267     st.subheader("📄 Overlay Information")
268     df_overlay = app_info.rename(columns=mappings["rename_general"])[overlays_col].T.
269         reset_index()
270     df_overlay = df_overlay.rename(columns={'index': "Overlay Detail", df_overlay.
271         columns[1]: "Values"})
272     df_overlay["Values"] = df_overlay["Values"].astype(str)
273
274     st.dataframe(df_overlay, use_container_width=True, hide_index=True)
275
276     coll, col2 = st.columns(2)
277
278     with coll:
279         st.subheader("Download Contract List")
280         st.download_button(
281             label="📄 Download Excel",
282             data=contractlist_bytes,
283             file_name=f"report_contractlist_{application_id}.xlsx",
284             mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
285         )
286
287     with col2:
288         st.subheader("Download Payment List")
289         st.download_button(
290             label="📄 Download Excel",
291             data=paymentlist_bytes,
292             file_name=f"report_paymentlist_{application_id}.xlsx",
293             mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
294         )
295
296     elif application_id in ntc_ids:
297
298         app_info = df[df['application_id_res'].astype(str) == application_id]

```

```

296
297 processed_data = process_data(df_ntc,application_id)
298
299 st.subheader("Application Information")
300
301 st.subheader("📄 ID")
302
303 app_info_transposed = app_info.rename(columns=mappings["rename_general"])[ids_col
304 ].T.reset_index()
305 app_info_transposed = app_info_transposed.rename(columns={'index': "Detail",
306 app_info_transposed.columns[1]: "Values"})
307 app_info_transposed["Values"] = app_info_transposed["Values"].astype(str) #
308 Convert to string
309
310 st.dataframe(app_info_transposed, use_container_width=True, hide_index=True)
311
312 st.subheader("📊 Feature Information")
313
314 st.dataframe(
315     processed_data["data_feature"]
316     .style
317     .background_gradient(subset=['percent'], cmap='Blues')
318     .format({
319         "score": "{:.2f}",
320         "max_score": "{:.2f}",
321         "min_score": "{:.2f}",
322         "percent": "{:.2f}%"
323     }),
324     use_container_width=True,
325     hide_index=True
326 )
327
328 st.dataframe(
329     processed_data["data_summary"]
330     .style
331     .background_gradient(subset=['percent_score'], cmap='Blues')
332     .format({
333         "score_total": "{:.2f}",
334         "max_score_total": "{:.2f}",
335         "min_score_total": "{:.2f}",
336         "percent_score": "{:.2f}%"
337     }),
338     use_container_width=True,
339     hide_index=True
340 )
341
342 st.subheader("📄 Overlay Information")
343 overlay_info_transposed = app_info.rename(columns=mappings["rename_general"])[
344 overlays_col].T.reset_index()
345 overlay_info_transposed = overlay_info_transposed.rename(columns={'index':
346 "Overlay Detail", overlay_info_transposed.columns[1]: "Values"})
347 overlay_info_transposed["Values"] = overlay_info_transposed["Values"].astype(str)
348 # Convert to string
349
350 st.dataframe(overlay_info_transposed, use_container_width=True, hide_index=True)
351
352 coll1, coll2 = st.columns(2)
353
354 with coll1:
355     st.subheader("Download Contract List")
356     st.download_button(
357         label="📄 Download Excel",
358         data=contractlist_bytes,
359         file_name=f"report_contractlist_{application_id}.xlsx",
360         mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
361     )

```

```
357         with col2:
358             st.subheader("Download Payment List")
359             st.download_button(
360                 label="📄 Download Excel",
361                 data=paymentlist_bytes,
362                 file_name=f"report_paymentlist_{application_id}.xlsx",
363                 mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
364             )
365
366
367     else:
368         st.warning("❌ Application ID not found!")
```